Спринт 2 «Аудит вимог і стратегія тестування»

Версія 1.0. Вересень 2026.

1. Шапка спринта

Параметр	Значення
Назва, тижні	Спринт 2 «Аудит вимог і стратегія тестування», W4–5; два тижні: заняття 1 (Sprint Planning), самостійна робота, заняття 2 (Demo, бліц, Retro).
Теми	Шість критеріїв якості вимог і аудит специфікації за чек-листами; сім рівнів тестування; 24 типи тестування; «Ціль» і «Опис» тесту; план тестування продукту.
Вхід → інкремент	SRS продукту версії 1.0 «для аудиту», результати спринта 1, каталог, шаблони → Досьє якості v2: «Реєстр дефектів вимог + виправлена SRS + План тестування».

2. Замовлення і контекст

> *«Замовник передав специфікацію вимог до продукту. Перевірте її якість до початку тестування і запропонуйте стратегію: на яких рівнях і якими типами тестів ми перевірятимемо продукт».*

Специфікація вимог (SRS) 1.0 містить 36–42 вимоги: функціональні (`F-10` , `F-20` , …), до інтерфейсу користувача (`GUI-10` , …), до бізнес-логіки модуля (`BL-10` , …) і нефункціональні (`P-` продуктивність, `S-` захищеність, `U-` юзабіліті, `D-` документація); розділи 1–2 (терміни, посилання, ролі, обмеження) теж підлягають аудиту. У документі 24 дефекти — по чотири на кожний критерій; перелік команда не отримує. SRS 1.1 стане основою тест-кейсів (спринт 3), юніт-тестів за розділом BL (спринт 4) і матриці простежуваності (спринт 5): пропущений дефект повернеться тест-кейсом, який неможливо виконати. Друга частина замовлення — план, як саме і чому саме так перевірятимуть продукт, а не перелік термінів.

3. Мета і результати навчання

Після спринта ви зможете:

- застосувати чек-листи шести критеріїв якості вимог до реального документа;
- класифікувати дефект вимоги за критерієм і обґрунтувати це пунктом чек-листа;
- запропонувати виправлення, яке усуває дефект і не породжує нового;
- вести реєстр дефектів вимог і журнал змін специфікації;
- розрізняти рівні і типи тестування, пояснити призначення 7 рівнів і 24 типів;

- сформулювати «Ціль» та «Опис» тесту для продукту і скласти план тестування з пріоритетами й обґрунтуванням незастосовних типів.

4. Теоретичний мінімум

Частина А. Якість вимог

4.1 Критерій тестування вимог

Критерій тестування вимог — ознака, на підставі якої оцінюється вимога. Специфікація — перший об'єкт тестування: дефект вимоги, знайдений до реалізації, виправляється правкою тексту, після — переробкою коду і тестів. За дефектною вимогою неможливо побудувати коректний тест-кейс: неоднозначна дає два «очікувані результати», неповна мовчить про помилкові дані, нетестопридатна не дає критерію «пройдено / не пройдено». Одна вимога може порушувати кілька критеріїв; у реєстрі вказують **головний** — той, порушення якого усуває запропоноване виправлення; різні виправлення — різні рядки.

4.2 Критерій 1. Однозначність (unambiguity)

Вимога має одне значення і читається всіма учасниками однаково. Чек-лист:

1. Кожна вимога має одне значення.
2. Кожна вимога чітка і зрозуміла.
3. Кожна вимога трактується лише в тому сенсі, який вкладав у неї автор.
4. Кожна вимога точна, читабельна і допускає єдину інтерпретацію.
5. Специфічна термінологія єдина й однаково розуміється замовником і розробниками.

Продукт	Дефектна вимога (пункт)	Виправлена вимога
«ШопЛайн»	«Після оформлення замовлення система надсилає користувачу повідомлення» — кому, яким каналом, з чим? (п. 1, 4)	«Після підтвердження оплати система надсилає покупцеві лист на email з профілю з номером замовлення, переліком позицій і сумою у гривнях».
«СтейНау»	«Гість може забронювати номер на кілька днів» — «кілька» невизначено; розділ 5 оперує «ночами», термін не визначено (п. 2, 5).	«Гість бронює номер тривалістю від 1 до 30 ночей»; термін «ніч» додано до розділу 1.3.

4.3 Критерій 2. Повнота (completeness)

Вимога містить усе потрібне для реалізації і перевірки, а набір вимог достатній для продукту. Чек-лист:

1. Вимога містить повну інформацію про те, що в ній описується.
2. Усі необхідні і важливі вимоги присутні, і їх достатньо для розроблення продукту.
3. Немає посилань на неіснуючі документи, рисунки, схеми.
4. Обумовлено всі можливі допустимі вхідні дані, включно з недопустимими.

5. Описано реакції застосунку на всі можливі помилки та виняткові ситуації.

Продукт	Дефектна вимога (пункт)	Виправлена вимога
«ФінКит»	«Клієнт вводить суму переказу і підтверджує операцію» — допустимі значення і реакція на суму понад залишок чи денний ліміт не описані (п. 4, 5).	«Сума — додатне число з двома знаками після коми. Якщо перевищує залишок — відмова «Недостатньо коштів»; якщо разом із переказаним сьогодні перевищує 50 000 грн — «Перевищено денний ліміт»; форма лишається заповненою».
«ЕдуПортал»	«Викладач оцінює завдання за шкалою з Додатка Б» — Додатка Б немає (п. 3).	«Викладач оцінює завдання цілим числом 0–100; переведення у шкалу ECTS визначає розділ 5».

4.4 Критерій 3. Несуперечливість (consistency)

Вимоги узгоджені між собою і з іншими документами. Чек-лист:

1. Вимога не містить частин, що суперечать одна одній.
2. Вимоги не суперечать одна одній; це стосується і вимог, викладених в інших документах.

Продукт	Дефектна вимога (пункт)	Виправлена вимога
«АвтоРейт»	«Голосувати може будь-який відвідувач; для збереження голосу користувач має бути авторизований» — частини вимоги суперечать (п. 1).	«Голосувати може лише авторизований користувач; неавторизованому показується посилання «Увійдіть, щоб голосувати»».
«Персонал+»	F: «Тривалість заявки на відпустку не перевищує баланс днів». GUI: «Поле «Тривалість» приймає до 30 днів; заявка зберігається» — інтерфейс дозволяє заборонене (п. 2).	GUI: «Якщо тривалість перевищує баланс, кнопка «Подати» неактивна, під полем — «Тривалість перевищує баланс: N днів»».

4.5 Критерій 4. Простежуваність (traceability)

Кожну вимогу можна однозначно назвати, знайти і пов'язати з тест-кейсами й дефектами. Чек-лист:

1. Вимоги унікальні і не повторюються.
2. Усі вимоги мають унікальний ідентифікатор, наприклад `GUI-312`.
3. Формат ідентифікатора дозволяє додавати і вилучати вимоги з будь-якого документа чи сховища без впливу на решту ідентифікаторів.
4. Формат ідентифікатора дозволяє розділяти вимогу або додавати вимогу нижчого рівня без впливу на решту ідентифікаторів.
5. Вимоги не перекриваються і не перетинаються за змістом.

Тому ID йдуть із кроком 10: між `F-30` і `F-40` вставляють `F-35`; вилучена вимога залишає «дірку», номери не зсуваються.

Продукт	Дефектна вимога (пункт)	Виправлення
«ШопЛайн»	«F-40. Покупець може додати товар у кошик зі сторінки товару» і «F-90. Зі сторінки товару покупець додає товар до кошика кнопкою «У кошик»» — одна вимога під двома ID (п. 1, 5).	Залишити `F-40` з повним формулюванням; `F-90` вилучити із записом у журналі змін, посилання замінити.
«СтейНау»	Рядок розділу 4 «Поле «Телефон» — +380 і 9 цифр» без ідентифікатора (п. 2).	Присвоїти вільний ID: наступний (`GUI-95`) або проміжний (`GUI-35` між `GUI-30` і `GUI-40`).

4.6 Критерій 5. Здійсненність (practicability)

Чек-лист з одного пункту: вимоги мають бути здійсненними з погляду наявних ресурсів — технічних, часових, фінансових, людських. Допоміжні питання: чи існує технологія; чи не залежить виконання від систем, які продукт не контролює; чи можливо це у строки і бюджет; чи не суперечить фізичним або правовим обмеженням.

Продукт	Дефектна вимога	Виправлена вимога
«ФінКит»	«Система гарантує зарахування переказу в інший банк протягом 1 с у будь-який час доби» — залежить від банку-одержувача і міжбанківської системи.	«Система передає переказ на виконання не пізніше ніж через 5 с; до підтвердження банком-одержувачем відображається статус «В обробці»».
«ЕдуПортал»	«Система автоматично оцінює есе без участі викладача з точністю 100 %» — недосяжно для вільного тексту.	«Система автоматично оцінює тести із закритими питаннями за ключем; відкриті відповіді оцінює викладач».

4.7 Критерій 6. Тестопридатність (testability)

Те, що міститься у вимозі, можна успішно протестувати з урахуванням наявних ресурсів і часу. Класична неперевірювана вимога: «Застосунок не повинен містити помилок». Практичний тест: чи можна написати тест-кейс з однозначним результатом «пройдено / не пройдено»? Якщо бракує числа, умови чи способу спостереження — вимога нетестопридатна. Слова-маркери: «без помилок», «інтуїтивно», «зручно», «надійно», «прийнятно».

Продукт	Дефектна вимога	Виправлена вимога
«Персонал+»	«Інтерфейс подання заявки на відпустку має бути інтуїтивно зрозумілим».	«Новий працівник без інструктажу подає заявку не довше ніж за 3 хв і не більше ніж з однією помилкою; перевірка на 5 учасниках, критерій — 4 з 5».
«АвтоРейт»	«Зміна пароля у профілі має бути безпечною» — що спостерігати, невідомо.	«Пароль — 8–64 символи, щонайменше одна цифра і одна літера; інакше повідомлення «Пароль не відповідає вимогам», зміни не зберігаються».

Критерії пов'язані: «швидко» порушує і однозначність, і тестопридатність; головним обирайте той, до якого веде виправлення.

4.8 Реєстр дефектів вимог

Реєстр — єдина таблиця команди; рядок: **№ — ID вимоги — критерій — опис дефекту — пропозиція виправлення — хто знайшов**. Правила: один рядок — один дефект; ID обов'язковий (якщо дефект у відсутності ID — «розд. 4, рядок 6, без ID», критерій «простежуваність»); опис називає пункт чек-листа і цитує фрагмент; виправлення — готове формулювання, а не «уточнити»; якщо потрібне рішення замовника (невідома межа), запропонуйте значення з позначкою «потребує підтвердження замовником» — це питання на демо; «хто знайшов» — з незалежних аркушів, за потреби двоє.

№	ID	Критерій	Опис дефекту	Пропозиція виправлення	Хто знайшов
1	`F-70`	Повнота (п. 5)	«Клієнт вводить суму переказу і підтверджує операцію» — реакція на суму понад залишок не описана.	«…Якщо сума перевищує залишок, система відхиляє операцію з повідомленням «Недостатньо коштів»».	О. Коваль, І. Литвин

4.9 Процедура командного аудиту

1. **Розподіл розділів.** Керівник призначає кожному 1–2 розділи так, щоб кожен розділ перевіряли щонайменше двоє незалежно (для чотирьох: розділ 3 — двоє; 4 + 1; 5 + 2; 6 + 3 повторно); фіксується у таблиці задач.
2. **Незалежна перевірка.** Кожен проходить свої розділи вимога за вимогою за всіма шістьма чек-листами і веде власний аркуш. Обговорювати до зведення заборонено: знайдене двома окремо — майже напевно дефект, знайдене одним — предмет обговорення.
3. **Зведення.** Тест-аналітик об'єднує аркуші, вилучає дублікати (зберігаючи обох авторів), позначає спірні рядки.
4. **Обговорення спірних.** Аудитор-опонент (у меншій команді — Скрам-майстер) як «адвокат замовника» вимагає пункт чек-листа і доказ, що виправлення усуває дефект; необґрунтовані рядки вилучаються — 16 доведених дефектів кращі за 24 сумнівних.
5. **SRS 1.1 з журналом змін.** До копії SRS вносяться прийняті правки; на початку — журнал «версія — дата — ID — було — стало — № дефекту — автор». Незмінені вимоги дослівні, ID не перенумеровуються.

Частина Б. Рівні і типи тестування

4.10 Етап (рівень) і тип

Етап — стадія якогось процесу; рівень тестування визначає, *коли* і *на якому об'єкті* виконується перевірка — від окремого модуля до системи в руках замовника. **Тип** — форма чогось; тип тестування визначає, *що саме* перевіряється — аспект якості. Рівень і тип незалежні: функціональне тестування виконують і на системному рівні, і при прийманні замовником; на системному рівні застосовують і GUI-, і навантажувальне тестування. «Smoke» — рівень, не тип; «функціональне» — тип, не рівень.

4.11 Сім рівнів тестування

Рівень	Що перевіряє	Хто	Коли	Приклад для продукту
1. Модульне (unit testing)	Правильність окремих модулів (процедур, функцій, бібліотек) незалежно від оточення і без взаємодії з іншими модулями; модуль ізольовано, замість інших модулів — заглушки (stubs) і симулятори.	Розробник, SDET	Під час написання коду	«ФінКит»: `LoanCalculator.monthlyPayment` для P = 100 000, r = 20, n = 12 дає суму за ануїтетною формулою.
2. Інтеграційне (integration, multi-unit testing)	Правильність взаємодії модулів деякого набору один з одним.	Розробник, інженер	У міру складання модулів	«ШопЛайн»: `Cart.subtotal()` → `volumeDiscount` → `shippingFee`.
3. Приймальне в тестування (acceptance into testing, smoke)	Що основні функції зібраної в ціле системи працюють і тестування можна проводити далі.	Інженер з тестування	На кожній збірці, перед основним тестуванням	«СтейНау»: сайт відкривається, номери відображаються, форма бронювання приймає дані, адмінпанель доступна.
4. Системне (system testing)	Правильність роботи системи в цілому, здатність розв'язувати задачі користувачів у різних ситуаціях; через інтерфейс користувача імітацією його дій.	Команда тестування	Після smoke; основний обсяг	«Персонал+»: HR створює працівника → заявка на відпустку → погодження → баланс зменшується.
5. Альфа (alpha testing)	Перша спроба використати систему за призначенням в умовах, наближених до реальної експлуатації; перевірити працездатність і поведінку в таких умовах.	Внутрішні користувачі, не розробники	Після системного, до випуску назовні	«ЕдуПортал»: викладачі кафедри тиждень ведуть реальний курс на тестовому сервері.

Рівень	Що перевіряє	Хто	Коли	Приклад для продукту
6. Бета (beta testing)	Роздача зацікавленим особам для пробної експлуатації: розширити коло людей до офіційного випуску, зібрати й усунути виявлені ними помилки, вдосконалити і, можливо, додати функціональність за побажаннями.	Зовнішні добровольці	Перед офіційним випуском	«АвтоРейт»: доступ 200 запрошеним автолюбителям з формою зворотного зв'язку.
7. Приймальне замовником (customer acceptance testing)	Тестування системи замовником або користувачем для прийняття її в експлуатацію.	Замовник	Перед підписанням акту	«ФінКит»: представник банку проходить сценарії з договору — відкриття рахунку, переказ, оплата рахунку.

4.12 Двадцять чотири типи тестування

Друга група — **тестування продуктивності (performance)**: перевірка використання ресурсів (пам'яті, часу, дискового простору) за різних умов навантаження і доступності ресурсів; включає багато видів, тож у плані вказують конкретний вид.

Тип (укр / англ)	Що перевіряє	Приклад
Група 1. Функціональні та експлуатаційні типи		
1. Функціональне (functional, feature)	Функції з розділу функціональних вимог: і те, що система повинна робити, і те, чого не повинна.	«ШопЛайн»: товар додається в кошик; неавторизований покупець не може додати в обране.
2. Графічного інтерфейсу (GUI)	Візуальне подання: елементи керування та їх поведінка, компоненти візуалізації даних, зовнішній вигляд, дотримання рекомендацій з оформлення.	«СтейНау»: поле дати заїзду приймає лише дд.мм.рррр і вміщує значення повністю.
3. Юзабіліті (usability)	Зручність для кінцевого користувача: ергономіка, простота, інтуїтивність; зрозумілість документації і повідомлень; логічність функцій та їх зв'язків.	«Персонал+»: меню не глибше трьох рівнів; повідомлення про помилку каже, що виправити.
4. Захищеності (security)	Алгоритми криптозахисту, автентифікація, схема захисту і розділення доступу, коректність прав, стійкість до зламу.	«ФінКит»: без входу сторінка рахунків за прямим URL недоступна.
5. Потоків даних (data flow)	Перетворення і передавання даних між компонентами; тест-кейси будуються за моделями потоків даних або транзакцій, отриманими з бізнес-логіки чи коду.	«ШопЛайн»: ціна з картки товару → кошик → знижка → інвойс без спотворень.
6. Інтерфейсів (interface)	Введення і виведення з різних пристроїв або програм — програмні й апаратні інтерфейси; не плутати з інтерфейсом користувача.	«Персонал+»: імпорт працівників із CSV; «АвтоРейт»: відповідь API у JSON.

Тип (укр / англ)	Що перевіряє	Приклад
7. Інсталяційне (installation)	Інсталяційний пакет: встановлення в різних режимах і конфігураціях, деінсталяція, доповнення і додаткові модулі, готовність до роботи.	«ЕдуПортал»: розгортання резервного екземпляра з образу Docker, оновлення плагіна.
8. Конфігураційне (configuration)	Робота системи і компонентів (програмних і апаратних) у різних конфігураціях і середовищах, з різними пристроями і платформами.	«СтейНау»: бронювання у Chrome, Firefox, Safari і на екрані 375 px.
9. Об'ємне (volume)	Робота з великими обсягами даних: чи працюють алгоритми і чи зберігають властивості; час і ресурси не враховуються.	«ШопЛайн»: каталог на 100 000 товарів — фільтри і сортування правильні.
10. Надійності (reliability)	Статистичні параметри (кількість збоїв, час без збоїв, напрацювання на відмову) і механізми надійності: відновлення даних, обробка помилок і винятків.	«ФінКит»: доба безперервних переказів без втрачених транзакцій.
11. Доступності (availability)	Здатність зберігати працездатність; резервування, гаряча заміна, підтримка працездатності.	«ФінКит»: збій одного з дубльованих серверів БД не зупиняє вхід.
12. Відновлення (failover / recovery)	Здатність зберігати і відновлювати працездатність і дані після збоїв і помилок.	«СтейНау»: після перезапуску сервера підтверджені бронювання на місці.
13. Інтернаціоналізації (internationalization, i18n)	Здатність працювати в різних мовних конфігураціях оточення і з різними мовами.	«АвтоРейт»: коментар з ї, є, ґ, апострофом і емодзі не спотворюється.
14. Локалізації (localization, l10n)	Правильність адаптації до конкретної мови й культури: переклад інтерфейсу і документації, шрифти, кодування, формати дат і сум; робота інших типів тестів для цієї локалізації.	«Персонал+»: українська версія — переклад повний, дати дд.мм.рррр, суми «1 234,56 грн».
15. Документації (documentation)	Наявність, точність, повнота, зрозумілість, технічна коректність, гіперпосилання, грамотність, оформлення.	«ШопЛайн»: усі пункти меню кабінету описано в довідці.
Група 2. Тестування продуктивності (performance)		
16. Навантажувальне (load)	Робота за різних умов навантаження від мінімального і типового до максимального; поступове збільшення або зниження: 1, 2, …, максимум користувачів.	«ШопЛайн»: 1 / 10 / 50 / 100 покупців оформлюють замовлення — час відповіді.
17. Стресове (stress / capacity)	Екстремальне навантаження, що веде до нестачі пам'яті чи диска, за великої кількості або швидкості запитів.	«ФінКит»: тисячі запитів за секунду — як деградує і чи коректно відмовляє.
18. Стрибкове (spike)	Різкі стрибки: секундне підвищення до максимуму і різке повернення до норми.	«ЕдуПортал»: 500 студентів одночасно натискають «Почати тест».
19. Порівняльне (benchmarking)	Вимірювання параметрів продуктивності і порівняння з еталоном: теоретичними оцінками, попередніми версіями, конкурентами.	«АвтоРейт»: час пошуку у версії 2 проти версії 1.

Тип (укр / англ)	Що перевіряє	Приклад
20. Профілювання (profiling)	Ті самі вимірювання без еталона; мета — отримати величини, що стануть еталоном або рекламною інформацією.	«СтейНау»: час розрахунку вартості 10 000 бронювань.
21. Стабільності (stability)	Стабільність показників протягом довгого періоду реальної або імітованої роботи.	«Персонал+»: швидкість обробки заявок після 72 год не змінилася.
22. Витоків пам'яті (memory leaks)	Пам'ять виділяється, але не звільняється; спожита пам'ять зростає до вичерпання.	«АвтоРейт»: тривала робота — порівняння пам'яті з оцінками розробників.
23. Вузьких місць (bottleneck)	Компоненти, де обробка повільніша за інші; аналіз за моделями потоків і діаграмами передавання даних.	«ФінКит»: найповільніший крок у «заявка → скоринг → рішення».
24. Умов гонок (contention, race conditions)	Результат залежить від черговості операцій у паралельних задачах при спільному використанні ресурсів.	«СтейНау»: два гості одночасно бронюють останній номер — підтверджується одне.

4.13 Як формулювати «Ціль» і «Опис»

Ціль — яка мета переслідується тестом і що має бути перевірено: об'єкт і ризик продукту, який закриває перевірка. **Опис** — яким чином виконуватиметься перевірка: дії, дані, інструменти, критерій успіху. «Ціль: перевірити навантаження. Опис: запустити навантажувальний тест» не приймається — воно однакове для будь-якого продукту.

Тип, продукт	Ціль	Опис
Навантажувальне, «ФінКит»	Перевірити, що час відповіді операції «Переказ між рахунками» при 1–200 одночасних клієнтах не перевищує 3 с і жоден переказ не втрачається — пік припадає на дні виплати зарплат.	Сценарій «вхід → переказ 100 грн → баланс» у JMeter або k6; сходинки 1 / 10 / 50 / 100 / 200 віртуальних користувачів по 5 хв; фіксуються 95-й перцентиль часу відповіді і частка помилок; сума балансів звіряється з початковою.
Захищеності, «Персонал+»	Переконатися, що працівник не може переглянути чи змінити дані іншого працівника (зарплата, РНОКПП) — це персональні дані.	Під обліковим записом працівника підставити в URL профілю ідентифікатор іншого; спробувати відкрити розділ адміністрування; очікується відмова (403 або перенаправлення).
Інтернаціоналізації, «АвтоРейт»	Перевірити, що коментарі з літерами ї, є, ґ, апострофом та емодзі зберігаються без спотворень — аудиторія україномовна.	Залишити коментар «Об'єм двигуна — ґрунтовний»; перевірити на сторінці моделі і після перезавантаження у Chrome і Firefox.

4.14 Структура плану тестування

План тестування (test plan) фіксує стратегію. Розділи: (1) продукт і межі тестування; (2) рівні — сім рядків, усі обов'язкові; (3) типи — не менше нормативу, з пріоритетами; (4) незастосовні типи з обґрунтуванням; (5) інструменти, середовище, тестові дані; (6) ризики і припущення. Розділи 2–4 — єдина таблиця:

Рівень / тип	Ціль	Опис	Пріоритет	Інструменти / дані	Застосовність
Рівень 3. Smoke	Переконатися перед кожним прогоном, що demo «ШопЛайн» живий: каталог, кошик, вхід, оформлення.	8 перевірок за чек-листом: головна, пошук, кошик, вхід покупця, крок «адреса».	High	Chrome; обліковий запис покупця з каталогу	Так
Тип 7. Інсталяційне	—	—	—	—	Ні: хмарний сервіс без інсталяційного пакета; розгортання резервного екземпляра — у межах конфігураційного.

Пріоритет визначає продукт: для «ФінКит» захищеність і потоки даних — High, локалізація — Low; для «АвтоРейт» умови гонок при голосуванні — High, доступність із резервуванням — Low. Незастосовність обґрунтовують природою продукту, а не браком часу.

5. Завдання спринта

5.1 Спільний обсяг робіт

1. Отримати SRS 1.0; розподілити розділи так, щоб кожен перевіряли щонайменше двоє.
2. Провести незалежний аудит за шістьма чек-листами; кожен веде власний аркуш.
3. Звести реєстр дефектів: не менше нормативу, ≥2 на кожний критерій; кожен рядок з ID, критерієм, описом, виправленням, автором.
4. Обговорити спірні, затвердити реєстр (`requirements-defects.xlsx`).
5. Підготувати SRS 1.1 з журналом змін (`SRS-1.1.docx` або `.md`).
6. Скласти план тестування (`test-plan.docx` або `.md`): 7 рівнів і не менше нормативу типів з Ціллю, Описом, пріоритетом, інструментами; незастосовні типи з обґрунтуванням.
7. Заповнити таблицю задач, провести ретро, здати протокол і peer-review форму.

5.2 Розбивка за ролями

Незалежний аудит своїх розділів виконує кожен незалежно від ролі; нижче — задачі понад це.

Роль	Задачі	Артефакти
Керівник тестування	Декомпозиція замовлення; розподіл розділів SRS; модерація зведення і обговорення спірних; затвердження плану і пріоритетів; питання замовнику; демо.	«Розподіл розділів» у таблиці задач; сценарій демо
Тест-аналітик	Зведення аркушів у реєстр, дедуплікація, класифікація за критеріями; SRS 1.1 і журнал змін.	`requirements-defects.xlsx` ; `SRS-1.1`
Інженер з тестування	План тестування: 7 рівнів і типи з Ціллю/Описом, пріоритетами, інструментами і тестовими даними demo-застосунку; незастосовні типи.	`test-plan`

Роль	Задачі	Артефакти
Скрам-майстер / рецензент	Таблиця задач, standup; перевірка, що кожен дефект прив'язаний до ID і критерію, а кожен рядок плану має Ціль і Опис, специфічні для продукту; протокол ретро; підготовка до бліцу.	Таблиця задач з позначками перевірки; протокол ретро
Аудитор-опонент (5 осіб)	«Адвокат замовника»: заперечує сумнівні дефекти (вимагає пункт чек-листа і доказ, що виправлення допомагає); перевіряє реалістичність плану — чи виконується кожен Опис на demo; формулює 5 очікуваних питань замовника.	Аркуш «Спірні» з рішеннями; список питань замовника

Для 3 осіб Скрам-майстер/рецензент поєднується з Керівником; кожен розділ SRS однаково перевіряють двоє.

6. Нормативи за розміром команди

Норматив	3 особи	4 особи	5 осіб
Знайдених дефектів SRS із 24 закладених	≥12	≥16	≥20
Типів тестування у плані; усі 7 рівнів — обов'язково для всіх	≥9	≥12	≥15
Дефектів на кожний із 6 критеріїв (з DoD)	≥2 — незалежно від розміру		

7. Definition of Done

- Знайдено не менше нормативу дефектів, щонайменше по 2 на кожний із 6 критеріїв.
- Кожен дефект прив'язаний до ID вимоги, має критерій, опис, виправлення і автора.
- SRS 1.1 містить журнал змін; кожна правка посилається на № дефекту; ID не перенумеровані.
- План містить усі 7 рівнів і не менше нормативу типів; для кожного — Ціль і Опис, специфічні для продукту, пріоритет та інструменти.
- Незастосовні типи перелічені з обґрунтуванням.
- Таблиця розподілу задач і протокол ретро здані.

8. Структура інкременту

```
docs/sprint2/
  requirements-defects.xlsx   аркуші «Реєстр» (№, ID, критерій, опис, виправлення, хто знайшов),
                              «Розподіл розділів», «Спірні» (рядок, аргументи, рішення)
  SRS-1.1.docx | SRS-1.1.md   журнал змін на початку; повний текст вимог із правками
  test-plan.docx | test-plan.md  розділи 1–6 за п. 4.14; таблиця «рівень/тип — Ціль — Опис — пріоритет — інструменти/дані — застосовність»
  task-board.xlsx             таблиця розподілу задач (або посилання на GitHub Projects)
  retro-sprint2.md            протокол ретро Start/Stop/Continue
```

Файли — у репозиторії команди на GitHub; Google Sheets/Docs допускаються за наявності посилань у `docs/sprint2/README.md`.

9. Хід спринта

Заняття 1 — Sprint Planning (~30 хв на команду)

- 0–5 хв: замовник озвучує замовлення, передає SRS 1.0.
- 5–12 хв: команда переглядає структуру SRS, ставить питання замовнику.
- 12–22 хв: декомпозиція в задачі, розподіл ролей і розділів SRS, фіксація у таблиці задач.
- 22–28 хв: строки незалежної перевірки, дата зведення, формат файлів.
- 28–30 хв: викладач затверджує таблицю розподілу.

Між заняттями (рекомендований графік, 12 днів)

- Дні 1–4: незалежна перевірка; standup «що зроблено / що блокує».
- День 5: сесія зведення (60 хв); дні 6–7: обговорення спірних, затвердження реєстру.
- Дні 6–9: SRS 1.1 з журналом змін (Тест-аналітик) паралельно з планом (Інженер).
- Дні 10–11: рецензування (Скрам-майстер, Аудитор-опонент), правки; день 12: репетиція демо, підготовка до бліцу.

Заняття 2 — Demo, бліц, Retro

- Демо 10 хв: 1 хв — контекст; 3 хв — реєстр: розподіл дефектів за критеріями і розділами, три показові дефекти (очевидний, спірний, найважливіший для наступних спринтів); 2 хв — SRS 1.1: журнал змін, що потребує рішення замовника; 3 хв — план: пріоритети, два приклади Ціль/Опис, незастосовні типи; 1 хв — питання замовника.
- Бліц: 1–2 питання довільному члену команди (для 5 осіб — двом); peer-review форма 5 хв; Retro 15 хв за шаблоном Start/Stop/Continue.

До здачі: `requirements-defects.xlsx`, `SRS-1.1`, `test-plan`, таблиця задач, протокол ретро, peer-review форми.

10. Оцінювання спринта

Складова	Бали	Критерії для цього інкременту
Повнота	2	2 — дефектів не менше нормативу і ≥2 на кожний критерій; усі 7 рівнів і не менше нормативу типів. 1 — одна з умов не виконана. 0 — обидві.
Коректність	2	2 — кожен дефект справді порушує названий критерій, виправлення усуває його і не породжує нового; Ціль і Опис відповідають суті типу. 1 — до чверті рядків з помилками класифікації або невдалими виправленнями. 0 — більше.
Обґрунтованість і документування	2	2 — план специфічний для продукту (Опис виконуваний на demo, названі інструменти і дані), пріоритети і незастосовні типи пояснені; журнал змін повний. 1 — частина рядків абстрактна. 0 — план не відрізняється від переліку термінів.

Складова	Бали	Критерії для цього інкременту
Виконання ролі	3	За артефактами ролі, таблицею задач і колонкою «хто знайшов»: внесок в аудит є у кожного.
Бліц	1	Правильна відповідь на питання з розділу 11 або банку.

Штраф −1 за незданий протокол ретро або таблицю задач; доздача протягом тижня — −20 % командної частини; бонус +0,5 за змістовне питання чи знайдену помилку на демо іншої команди.

11. Бліц-питання спринта

1. Що таке критерій тестування вимог?
2. Назвіть шість критеріїв якості вимог.
3. Який критерій порушує вимога «Пошук має працювати швидко» і чому саме він головний?
4. Чим неповна вимога відрізняється від неоднозначної? Наведіть по прикладу.
5. Що перевіряє критерій простежуваності і навіщо ідентифікатори йдуть із кроком 10?
6. Наведіть приклад нетестопридатної вимоги і виправте її.
7. Чим рівень тестування відрізняється від типу? До чого належить «smoke»?
8. Чим приймальне в тестування (smoke) відрізняється від приймального замовником?
9. У чому різниця між навантажувальним, стресовим і стрибковим тестуванням?
10. Чим інтернаціоналізація відрізняється від локалізації?
11. Чим GUI-тестування відрізняється від тестування інтерфейсів?
12. Чим порівняльне тестування (benchmarking) відрізняється від профілювання? Що таке заглушка і на якому рівні її використовують?

12. Типові помилки й поради

- **Дефект без ID** («десь у розділі 3») не зараховується: реєстр має простежуватись до вимоги.
- **Критерій «за відчуттям».** Аргументуйте пунктом чек-листа: виправлення додає число — тестопридатність, уточнює об'єкт — однозначність.
- **Виправлення, що породжує новий дефект:** «швидко» → «дуже швидко»; посилання на неіснуючий документ; нова вимога без ID.
- **Перекіс реєстру:** 20 дефектів однозначності і жодного здійсненності. Несуперечливість і здійсненність шукайте окремо, порівнюючи F, GUI і BL між собою.
- **Абстрактні Ціль/Опис** («перевірити навантаження») не відрізняють ваш продукт від будь-якого іншого.
- **Механічний план:** усі 24 типи «застосовуються» без пріоритетів або половина «не застосовується» без причини.
- **Плутанина рівень/тип:** smoke — рівень, функціональне — тип; у плані вони в різних рядках.

- **Аудит «гуртом»** замість незалежної перевірки губить перевагу командного рецензування; заповнюйте «хто знайшов» чесно — це основа індивідуальної оцінки.